

Laboratorio: API REST con Flask + Supabase (PostgreSQL)

Row Level Security (RLS) · Autenticación JWT · Documentación Swagger ·
Despliegue gratuito

Contenido

1. ¿Qué vamos a construir?	4
2. Conceptos clave en 5 minutos.....	4
3. Requisitos.....	5
Parte A: preparar Supabase	5
Parte B: crear el proyecto en tu computadora.....	7
Parte C: el código fuente, archivo por archivo	10
Parte D: ejecutar y probar con Swagger	28
Parte E: demostrar RLS (usuario A vs. usuario B)	33
Parte F: publicar gratis en Render (o Railway)	37
Errores frecuentes y soluciones.....	38
Retos para practicar	38
Entregables del laboratorio	38

Objetivo: construir, probar y publicar gratis una API REST de *tareas* donde cada usuario solo puede ver y modificar **sus propias** tareas. La seguridad no depende solo del código Python: la aplica **PostgreSQL** con **Row Level Security (RLS)**, a partir del **token JWT** del usuario.

API de Tareas (Flask + Supabase) 1.0.0 OAS 3.0

/openapi.json

API REST de laboratorio: **Flask + Blueprints + Supabase (PostgreSQL)** con **RLS** y autenticación **JWT**.

Cómo probarla:

1. **POST** `/api/auth/registro` para crear un usuario.
2. **POST** `/api/auth/login` y copia el `access_token`.
3. Pulsa el botón **Authorize** y pega el token (sin la palabra Bearer).
4. Usa los endpoints de `/api/tareas`.

Authorize

salud

Estado del servicio.

GET

/api/salud

Comprobar que la API está en línea

auth

Registro, inicio de sesión y tokens JWT (Supabase Auth).

POST

/api/auth/registro

Registrar un nuevo usuario

POST

/api/auth/login

Iniciar sesión

POST

/api/auth/refresh

Renovar el access_token

GET

/api/auth/perfil

Ver mi perfil (contenido del JWT)

tareas

CRUD de tareas protegido con JWT y Row Level Security (RLS).

GET

/api/tareas

Listar mis tareas

POST

/api/tareas

Crear una tarea

GET

/api/tareas/{tarea_id}

Ver una tarea por su id

PATCH

/api/tareas/{tarea_id}

Actualizar una tarea

DELETE

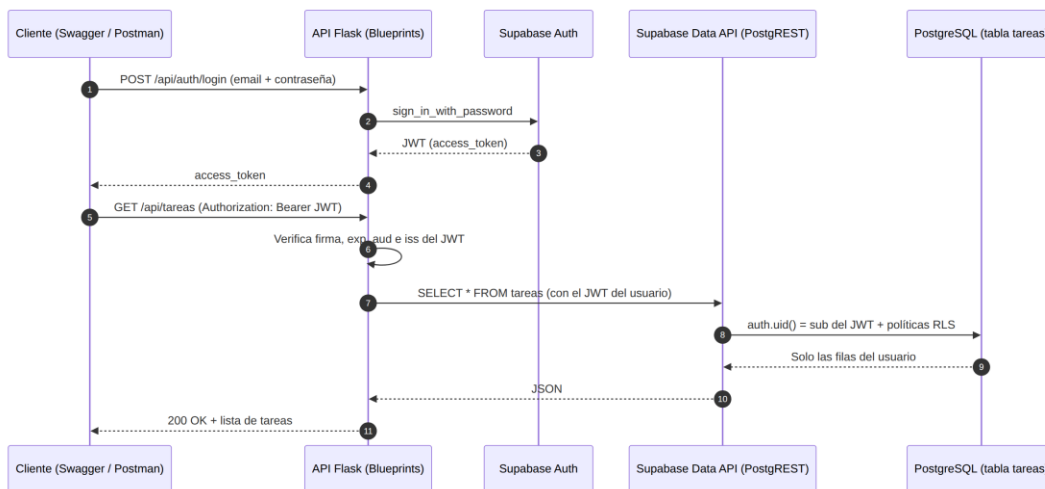
/api/tareas/{tarea_id}

Eliminar una tarea

Schemas

Swagger de la API

1. ¿Qué vamos a construir?



Método	Ruta	¿Protegida?	Descripción
GET	/api/salud	No	Comprueba que la API está en línea
POST	/api/auth/registro	No	Crea un usuario
POST	/api/auth/login	No	Inicia sesión y devuelve el JWT
POST	/api/auth/refresh	No	Renueva el JWT con el refresh_token
GET	/api/auth/perfil	Sí	Muestra el contenido de tu JWT
GET	/api/tareas	Sí	Lista tus tareas (?completada=true&prioridad=alta)
POST	/api/tareas	Sí	Crea una tarea
GET	/api/tareas/{id}	Sí	Ve una tarea
PATCH	/api/tareas/{id}	Sí	Actualiza campos de una tarea
DELETE	/api/tareas/{id}	Sí	Elimina una tarea
GET	/docs	No	Documentación interactiva Swagger UI
GET	/openapi.json	No	Especificación OpenAPI 3

Tecnologías: Python 3.12 · Flask 3 · flask-smorest (Blueprints + OpenAPI/Swagger + validación con Marshmallow) · Supabase (PostgreSQL + Auth) · PyJWT · Gunicorn · Render/Railway.

2. Conceptos clave en 5 minutos

API REST: un servidor que expone *recursos* (aquí, **tareas**) mediante URL y verbos HTTP: **GET** (leer), **POST** (crear), **PATCH** (modificar), **DELETE** (eliminar). Responde en JSON con códigos de estado: **200** OK, **201** creado, **401** no autenticado, **404** no encontrado, **422** datos inválidos.

Blueprint (Flask): un “módulo” de rutas. En lugar de escribir todo en un archivo gigante, separamos la API en piezas: **salud**, **auth** y **tareas**. Cada Blueprint tiene su prefijo de URL (**/api/auth**, **/api/tareas**).

JWT (JSON Web Token): una “credencial firmada” que el servidor de autenticación entrega al usuario tras el login. Tiene tres partes **ENCABEZADO.CARGA.FIRMA** codificadas en Base64URL. La carga contiene *claims* como **sub** (id del usuario), **email**, **role** y **exp** (fecha de expiración).

Cualquiera puede **leer** un JWT (¡no guardes secretos en él!), pero nadie puede **modificarlo** sin invalidar la firma. Pruébalo pegando tu token en <https://jwt.io>.

RLS (Row Level Security): una función de PostgreSQL que agrega automáticamente un filtro a cada consulta según **quién** la ejecuta. Nuestra política dice: *“solo puedes ver/modificar filas donde `user_id = auth.uid()`”*. `auth.uid()` es una función de Supabase que lee el `sub` del JWT. Resultado: aunque el código haga `SELECT * FROM tareas`, cada usuario solo recibe sus filas.

Swagger / OpenAPI: OpenAPI es un estándar para describir una API en JSON; Swagger UI es la página web que lo convierte en documentación interactiva desde la que puedes ejecutar peticiones. flask-smorest lo genera **automáticamente** a partir del código.

□ **Defensa en profundidad:** la API verifica el JWT (primera barrera) y PostgreSQL aplica RLS (segunda barrera). Si mañana alguien olvida un filtro en Python, la base de datos sigue protegiendo los datos.

3. Requisitos

- **Python 3.11 o superior** (recomendado 3.12). Verifica con `python --version` (Windows) o `python3 --version` (macOS/Linux).
- **Git** y una cuenta en **GitHub**.
- Un editor (VS Code recomendado).
- Cuenta gratuita en **Supabase** (<https://supabase.com>) y en **Render** (<https://render.com>). Ambas permiten registrarse con GitHub y **no piden tarjeta**.

Parte A: preparar Supabase

A.1 Crear el proyecto

1. Entra a <https://supabase.com/dashboard> e inicia sesión con GitHub.
2. Pulsa **New project**. Elige un nombre (por ejemplo `lab-api-tareas`), una contraseña para la base de datos (guárdala) y la región más cercana (por ejemplo *South America (São Paulo)*).
3. Espera 1 o 2 minutos hasta que el proyecto esté listo.

A.2 Desactivar la confirmación por correo (solo para el laboratorio)

Por defecto Supabase exige confirmar el correo antes de iniciar sesión. Para poder probar con correos ficticios:

1. Menú lateral **Authentication** → **Sign In / Providers** (o **Providers**) → **Email**.
2. Desactiva **Confirm email** y guarda.

□ En un sistema real **debe** estar activado. Lo desactivamos solo para practicar.

A.3 Crear la tabla y las políticas RLS

1. Menú lateral **SQL Editor** → **New query**.
2. Pega el contenido completo del archivo `sql/01_esquema_y_rls.sql` (está abajo) y pulsa **Run**.
3. Al final verás `rowsecurity = true` y las 4 políticas creadas.

□ **Archivo:** `sql/01_esquema_y_rls.sql`

```

-- =====
-- 01_esquema_y_rls.sql
-- Tabla "tareas" + Row Level Security (RLS)
--
-- CÓMO USARLO: Supabase > SQL Editor > New query > pega TODO > Run.
-- Se puede ejecutar varias veces sin errores (es idempotente).
-- =====

-- 1) TABLA -----
create table if not exists public.tareas (
  id          uuid primary key default gen_random_uuid(),
  -- Dueño de la tarea. Por defecto es el usuario que hace la petición.
  user_id     uuid not null default auth.uid()
              references auth.users (id) on delete cascade,
  titulo      text not null check (char_length(titulo) between 3 and 120),
  descripcion  text check (descripcion is null or char_length(descripcion) <= 500),
  prioridad   text not null default 'media' check (prioridad in ('baja', 'media',
'alta')),
  completada  boolean not null default false,
  creado_en   timestampz not null default now()
);

comment on table public.tareas is 'Tareas personales de cada usuario (protegidas con RLS).';

-- Índice para que las políticas RLS (que filtran por user_id) sean rápidas.
create index if not exists tareas_user_id_idx on public.tareas (user_id);

-- 2) PERMISOS BÁSICOS -----
-- Los usuarios anónimos (sin iniciar sesión) NO pueden tocar la tabla.
revoke all on public.tareas from anon;
-- Los usuarios autenticados pueden intentar operar... pero RLS decide QUÉ filas.
grant select, insert, update, delete on public.tareas to authenticated;

-- 3) ACTIVAR ROW LEVEL SECURITY -----
-- Con RLS activo y SIN políticas, nadie ve nada. Las políticas abren el acceso.
alter table public.tareas enable row level security;

-- 4) POLÍTICAS -----
-- Se borran primero para poder volver a ejecutar este script sin errores.
drop policy if exists "tareas_select propias" on public.tareas;
drop policy if exists "tareas_insert propias" on public.tareas;
drop policy if exists "tareas_update propias" on public.tareas;
drop policy if exists "tareas_delete propias" on public.tareas;

-- SELECT: solo veo las filas donde yo soy el dueño.
create policy "tareas_select propias" on public.tareas
  for select to authenticated
  using ( (select auth.uid()) = user_id );

-- INSERT: solo puedo crear filas a mi nombre.
create policy "tareas_insert propias" on public.tareas
  for insert to authenticated
  with check ( (select auth.uid()) = user_id );

-- UPDATE: solo modifico mis filas y no puedo "regalarlas" a otro usuario.
create policy "tareas_update propias" on public.tareas
  for update to authenticated
  using ( (select auth.uid()) = user_id )

```

```

with check ( (select auth.uid()) = user_id );

-- DELETE: solo borro mis filas.
create policy "tareas_delete_propias" on public.tareas
  for delete to authenticated
  using ( (select auth.uid()) = user_id );

-- 5) COMPROBACIÓN -----
-- Debe mostrar rowsecurity = true
select tablename, rowsecurity from pg_tables where schemaname = 'public' and tablename =
'tareas';
-- Debe mostrar las 4 políticas
select policyname, cmd from pg_policies where schemaname = 'public' and tablename =
'tareas' order by policyname;

```

¿Qué hace cada parte?

- `user_id uuid not null default auth.uid()`: el dueño de la tarea se asigna **solo**, con el id del usuario del JWT. El cliente no lo envía.
- `references auth.users (id) on delete cascade`: si se borra un usuario, se borran sus tareas.
- `revoke all ... from anon`: quien no inició sesión no puede ni intentar consultar la tabla.
- `enable row level security`: activa RLS. **Sin políticas, nadie vería nada.**
- `using (...)`: condición para **ver/afectar** filas existentes (SELECT, UPDATE, DELETE).
- `with check (...)`: condición que deben cumplir las filas **nuevas o modificadas** (INSERT, UPDATE).
- `(select auth.uid())` entre paréntesis: recomendación de Supabase para que PostgreSQL calcule el valor una sola vez por consulta (mejor rendimiento).

A.4 Copiar las credenciales

1. Ve a **Project Settings** (ícono de engranaje) → **Data API** (o **API**): copia la **Project URL**, por ejemplo `https://abcdefghijklmnop.supabase.co`.
2. En **Project Settings** → **API Keys**: copia la clave **publishable** (`sb_publishable_...`) o, en la pestaña *Legacy API keys*, la clave **anon public** (`eyJ...`). Cualquiera de las dos sirve.
3. (Opcional) En **Project Settings** → **JWT Keys** verás si tu proyecto firma los tokens con claves asimétricas (ES256/RS256, lo habitual en proyectos nuevos) o con el secreto *Legacy JWT secret* (HS256). **No necesitas configurar nada**: la API detecta el algoritmo por sí sola.

❑ **Nunca** uses la clave `service_role` / `secret` en este laboratorio: esa clave **ignora RLS** y tendría acceso total a los datos.

Parte B: crear el proyecto en tu computadora

B.1 Estructura final del proyecto

```

api-tareas-flask/
├── app/
│   ├── __init__.py      ← fábrica de la aplicación (create_app)
│   ├── config.py        ← configuración y variables de entorno
│   ├── supabase_client.py ← clientes de Supabase (público y por usuario)
│   ├── errores.py        ← manejadores globales de errores
│   └── salud/
│       └── __init__.py

```

├── routes.py	← Blueprint /api/salud
├── auth/	
│ ├── __init__.py	
│ ├── jwt_utils.py	← verificación del JWT
│ ├── decorators.py	← decorador @token_requerido
│ ├── schemas.py	← validación/documentación de auth
│ └── routes.py	← Blueprint /api/auth
├── tareas/	
│ ├── __init__.py	
│ ├── schemas.py	← validación/documentación de tareas
│ └── routes.py	← Blueprint /api/tareas (CRUD)
├── sql/	
│ ├── 01_esquema_y_rls.sql	
│ └── 02_probar_rls_en_sql_editor.sql	
├── scripts/	
│ └── demo_rls.py	← demostración automática de RLS
├── tests/	
│ ├── __init__.py	
│ └── test_api.py	← pruebas automáticas con pytest
├── .env.example	
├── .gitignore	
├── .python-version	
├── Procfile	← comando de inicio (Railway)
├── railway.json	← configuración de Railway
├── render.yaml	← configuración de Render (opcional)
├── pytest.ini	
├── requirements.txt	
├── requirements-dev.txt	
└── wsgi.py	← punto de entrada

B.2 Crear la carpeta y el entorno virtual

Un **entorno virtual** aísla las librerías de este proyecto de las del resto de tu computadora.

Windows (PowerShell o CMD):

```
mkdir api-tareas-flask
cd api-tareas-flask
python -m venv venv
venv\Scripts\activate
```

Si PowerShell muestra un error de *ejecución de scripts deshabilitada*, ejecuta una sola vez: `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` y vuelve a activar.

macOS / Linux:

```
mkdir api-tareas-flask
cd api-tareas-flask
python3 -m venv venv
source venv/bin/activate
```

Sabrás que está activo porque la terminal muestra `(venv)` al inicio.

B.3 Crear las carpetas

Windows (PowerShell):

```
mkdir app, app\salud, app\auth, app\tareas, sql, scripts, tests
ni app\salud\__init__.py, app\auth\__init__.py, app\tareas\__init__.py, tests\__init__.py
```

macOS / Linux:

```
mkdir -p app/salud app/auth app/tareas sql scripts tests
touch app/salud/__init__.py app/auth/__init__.py app/tareas/__init__.py tests/__init__.py
```

Los archivos `__init__.py` vacíos le indican a Python que esas carpetas son **paquetes** importables.

B.4 Dependencias

Crea `requirements.txt`:

□ **Archivo:** `requirements.txt`

```
# Dependencias del proyecto (versiones probadas juntas)
Flask==3.1.3
flask-smorest==0.47.0
marshmallow==4.3.1
flask-cors==6.0.5
supabase==2.31.0
PyJWT[crypto]==2.12.1
python-dotenv==1.2.2
gunicorn==26.2.0
```

Librería	¿Para qué sirve?
Flask	Framework web
flask-smorest	Blueprints con OpenAPI/Swagger automático y validación
marshmallow	Define y valida los esquemas de datos (JSON)
flask-cors	Permite que un frontend de otro dominio consuma la API
supabase	Cliente oficial de Supabase (Auth + base de datos)
PyJWT[crypto]	Verificar la firma de los JWT (HS256, ES256, RS256)
python-dotenv	Leer el archivo <code>.env</code>
gunicorn	Servidor de producción (lo usa Render/Railway)

Crea también `requirements-dev.txt` (para las pruebas):

□ **Archivo:** `requirements-dev.txt`

```
# Solo para desarrollo y pruebas
-r requirements.txt
pytest==8.4.2
```

Instala todo:

```
pip install -r requirements-dev.txt
```

B.5 Variables de entorno

Crea `.env.example` (plantilla que **sí** se sube a GitHub):

□ **Archivo:** `.env.example`

```
# Copia este archivo como .env y completa los valores de TU proyecto Supabase.
# Supabase > Project Settings > API (Data API / API Keys)

# URL del proyecto (sin barra final)
SUPABASE_URL=https://TU-PROYECTO.supabase.co
```

```
# Clave pública: "anon" (legacy, empieza con eyJ...) o "publishable" (sb_publishable...)
SUPABASE_KEY=pega-aqui-tu-clave-publica

# OPCIONAL: solo si tu proyecto firma los JWT con HS256 (legacy JWT secret).
# Si lo dejas vacío, la API verifica con las claves públicas (JWKS) o consulta a Supabase Auth.
SUPABASE_JWT_SECRET=
```

Luego **copia** ese archivo como `.env` y reemplaza los valores con los de tu proyecto (paso A.4):

```
# Windows
copy .env.example .env
# macOS / Linux
cp .env.example .env
```

Crea `.gitignore` para que el `.env` (con tus claves) **nunca** llegue a GitHub:

❑ **Archivo:** `.gitignore`

```
# Entorno virtual
venv/
.env/
# Variables secretas
.env
# Python
__pycache__/
*.pyc
.pytest_cache/
# Editores / SO
.vscode/
.idea/
.DS_Store
```

Parte C: el código fuente, archivo por archivo

Escribe (o copia) cada archivo en la ruta indicada. Lee los comentarios: explican el **porqué** de cada línea importante.

C.1 `app/config.py`: configuración

Centraliza la configuración. Lee las claves de Supabase desde variables de entorno y define cómo se genera la documentación Swagger (`/docs`) y OpenAPI (`/openapi.json`).

❑ **Archivo:** `app/config.py`

```
"""
config.py
-----
Lee la configuración desde variables de entorno.

En tu computadora las variables vienen del archivo .env (gracias a python-dotenv).
En Render o Railway se configuran en el panel web ("Environment Variables").
¡Nunca subas el archivo .env a GitHub!
"""
import os

from dotenv import load_dotenv
```

```

# Carga el archivo .env (si existe) en las variables de entorno del proceso.
load_dotenv()

class Config:
    # --- Supabase -----
    # URL del proyecto, por ejemplo: https://abcdefghijklmnop.supabase.co
    SUPABASE_URL = os.getenv("SUPABASE_URL", "").strip().rstrip("/")

    # Clave PÚBLICA del proyecto: "anon" (legacy) o "publishable" (sb_publishable...).
    # Es segura para usar en el backend porque la seguridad real la pone RLS.
    SUPABASE_KEY = os.getenv("SUPABASE_KEY", "").strip()

    # (Opcional) Secreto JWT "Legacy" (HS256). Solo se usa si tu proyecto
    # firma los tokens con HS256. Si tu proyecto usa las nuevas claves
    # asimétricas (ES256/RS256) NO lo necesitas.
    SUPABASE_JWT_SECRET = os.getenv("SUPABASE_JWT_SECRET", "").strip()

    # --- flask-smorest / OpenAPI / Swagger -----
    API_TITLE = "API de Tareas (Flask + Supabase)"
    API_VERSION = "1.0.0"
    OPENAPI_VERSION = "3.0.3"
    OPENAPI_URL_PREFIX = "/"
    OPENAPI_JSON_PATH = "openapi.json"          # especificación en /openapi.json
    OPENAPI_SWAGGER_UI_PATH = "/docs"          # interfaz Swagger en /docs
    OPENAPI_SWAGGER_UI_URL = "https://cdn.jsdelivr.net/npm/swagger-ui-dist@5/"
    OPENAPI_SWAGGER_UI_CONFIG = {"persistAuthorization": True}
    API_SPEC_OPTIONS = {
        "info": {
            "description": (
                "API REST de laboratorio: **Flask + Blueprints + Supabase (PostgreSQL)**"
                "con **RLS** y **autenticación JWT**.\n\n"
                "**Cómo probarla:**\n"
                "1. `POST /api/auth/registro` para crear un usuario.\n"
                "2. `POST /api/auth/login` y copia el `access_token`.\n"
                "3. Pulsa el botón **Authorize** y pega el token (sin la palabra"
                "Bearer).\n"
                "4. Usa los endpoints de `/api/tareas`."
            )
        }
    }

    # Mantiene el orden de los campos JSON tal como se definen.
    JSON_SORT_KEYS = False

class TestConfig(Config):
    """Configuración usada por las pruebas automáticas (pytest)."""
    TESTING = True
    SUPABASE_URL = "https://proyecto-prueba.supabase.co"
    SUPABASE_KEY = "clave-publica-de-prueba"
    SUPABASE_JWT_SECRET = "secreto-de-prueba-con-al-menos-32-caracteres!!"

```

C.2 app/supabase_client.py: clientes de Supabase

Aquí está el **corazón de RLS** en el lado de Python: para leer y escribir datos creamos un cliente que envía el **JWT del usuario**. Supabase ejecuta la consulta *como ese usuario* y PostgreSQL aplica las políticas.

□ **Archivo:** app/supabase_client.py

```
"""
supabase_client.py
-----
Funciones para crear clientes de Supabase.

IDEA CLAVE (¡importante para entender RLS!):
* Creamos un cliente NUEVO en cada petición. Nunca compartimos un cliente
  global "con sesión", porque si el usuario A inicia sesión en un cliente
  compartido, el usuario B podría terminar usando el token de A.
* Para leer o escribir datos usamos el token JWT DEL USUARIO. Así PostgreSQL
  sabe quién hace la consulta (auth.uid()) y las políticas RLS filtran las
  filas automáticamente.
"""

from flask import current_app
from supabase import Client, ClientOptions, create_client

def _opciones(headers: dict | None = None) -> ClientOptions:
    # En un backend no queremos guardar sesiones ni refrescar tokens en segundo plano.
    return ClientOptions(
        headers=headers or {},
        auto_refresh_token=False,
        persist_session=False,
    )

def cliente_publico() -> Client:
    """Cliente con la clave pública. Se usa para registro, login y refresh."""
    return create_client(
        current_app.config["SUPABASE_URL"],
        current_app.config["SUPABASE_KEY"],
        options=_opciones(),
    )

def cliente_de_usuario(token: str) -> Client:
    """
    Cliente que actúa EN NOMBRE del usuario autenticado.

    Envía el encabezado Authorization: Bearer <token_del_usuario>
    Supabase (PostgREST) valida ese JWT y ejecuta la consulta con el rol
    'authenticated'; auth.uid() devuelve el id del usuario y RLS se aplica.
    """
    return create_client(
        current_app.config["SUPABASE_URL"],
        current_app.config["SUPABASE_KEY"],
        options=_opciones({"Authorization": f"Bearer {token}"})
    )
```

? ¿Por qué un cliente nuevo por petición? Un servidor atiende a muchos usuarios a la vez. Si compartiéramos un único cliente “con sesión”, el token de un usuario podría filtrarse a la petición de otro.

C.3 `app/auth/jwt_utils.py`: verificar el JWT

Comprueba que el token: (1) esté bien formado, (2) tenga una **firma válida**, (3) no esté **vencido** (`exp`), (4) sea para usuarios autenticados (`aud`) y (5) lo haya emitido **tu** proyecto (`iss`).

□ **Archivo:** `app/auth/jwt_utils.py`

```
"""
jwt_utils.py
-----
Verificación de Los tokens JWT que emite Supabase Auth.

Un JWT tiene 3 partes separadas por puntos:  ENCABEZADO.CARGA.FIRMA
* ENCABEZADO: algoritmo de firma (alg) y el id de la clave (kid).
* CARGA (payload / claims): sub (id del usuario), email, role, exp (expiración)...
* FIRMA: garantiza que nadie modificó el token.

Supabase puede firmar Los tokens de dos formas:
1. Claves ASIMÉTRICAS (ES256 o RS256) -> proyectos nuevos. Verificamos La firma
   con la clave PÚBLICA publicada en:  <SUPABASE_URL>/auth/v1/.well-known/jwks.json
2. Secreto compartido HS256 (legacy) -> proyectos antiguos. Verificamos con
   SUPABASE_JWT_SECRET; si no lo configuraste, Le preguntamos a Supabase Auth.
"""

import jwt
from flask import current_app
from jwt import PyJWKClient

from app.supabase_client import cliente_publico


class TokenInvalido(Exception):
    """Se lanza cuando el token no es válido, está vencido o mal formado."""

# Caché de clientes JWKS (uno por URL de proyecto) para no descargar
# las claves públicas en cada petición.
_clientes_jwks: dict[str, PyJWKClient] = {}


def _cliente_jwks() -> PyJWKClient:
    url = current_app.config["SUPABASE_URL"]
    if url not in _clientes_jwks:
        _clientes_jwks[url] = PyJWKClient(
            f"{url}/auth/v1/.well-known/jwks.json",
            headers={"apikey": current_app.config["SUPABASE_KEY"]},
            cache_jwk_set=True,
            lifespan=600, # segundos que se guardan las claves en memoria
        )
    return _clientes_jwks[url]


def verificar_token(token: str) -> dict:
    """
    Verifica el token y devuelve sus claims (la carga útil) si es válido.
    Si no es válido lanza TokenInvalido con un mensaje entendible.
    """
```

```

"""
url = current_app.config["SUPABASE_URL"]
reglas = {
    "audience": "authenticated",    # Supabase emite tokens con aud="authenticated"
    "issuer": f"{url}/auth/v1",      # y con iss="<SUPABASE_URL>/auth/v1"
    "leeway": 10,                    # tolerancia de 10 s por diferencias de reloj
    "options": {"require": ["exp", "sub", "aud", "iss"]},
}

try:
    encabezado = jwt.get_unverified_header(token)
except jwt.DecodeError:
    raise TokenInvalido("El token está mal formado.")

algoritmo = encabezado.get("alg")

try:
    # --- Caso 1: claves asimétricas (proyectos nuevos) -----
    if algoritmo in ("ES256", "RS256"):
        clave_publica = _cliente_jwks().get_signing_key_from_jwt(token).key
        return jwt.decode(token, clave_publica, algorithms=[algoritmo], **reglas)

    # --- Caso 2: secreto compartido HS256 (Legacy) -----
    if algoritmo == "HS256":
        secreto = current_app.config.get("SUPABASE_JWT_SECRET")
        if secreto:
            return jwt.decode(token, secreto, algorithms=["HS256"], **reglas)

    # Sin secreto: primero revisamos vencimiento, audiencia y emisor...
    claims = jwt.decode(
        token,
        options={"verify_signature": False, "require": ["exp", "sub"]},
        audience="authenticated",
        issuer=f"{url}/auth/v1",
        leeway=10,
    )
    # ...y Luego Le pedimos a Supabase Auth que confirme que el token es real.
    try:
        respuesta = cliente_publico().auth.get_user(token)
    except Exception:
        raise TokenInvalido("Supabase rechazó el token.")
    if not respuesta or not respuesta.user or respuesta.user.id != claims["sub"]:
        raise TokenInvalido("Supabase rechazó el token.")
    return claims

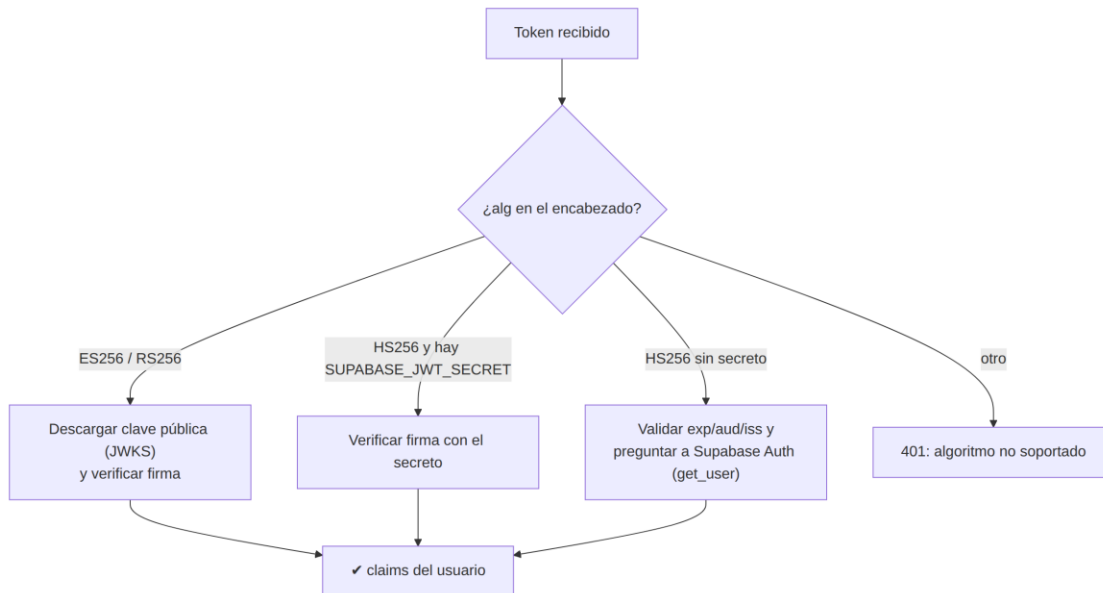
    raise TokenInvalido(f"Algoritmo de firma no soportado: {algoritmo}")

except jwt.ExpiredSignatureError:
    raise TokenInvalido("El token expiró. Vuelve a iniciar sesión o usa
/api/auth/refresh.")
except jwt.InvalidAudienceError:
    raise TokenInvalido("El token no es de un usuario autenticado (audiencia
incorrecta).")
except jwt.InvalidIssuerError:
    raise TokenInvalido("El token fue emitido por otro proyecto de Supabase.")
except jwt.PyJWKClientError:
    raise TokenInvalido("No se encontró la clave pública para verificar el token.")

```

```
except jwt.InvalidTokenError:
    raise TokenInvalido("La firma del token no es válida.")
```

Flujo de verificación:



C.4 app/auth/decorators.py: el decorador @token_requerido

Un **decorador** “envuelve” una función para ejecutar código antes que ella. Aquí extraemos el token del encabezado `Authorization: Bearer <token>`, lo verificamos y lo guardamos en `g` (un objeto que vive solo durante la petición actual).

□ **Archivo:** app/auth/decorators.py

```
"""
decorators.py
-----
Decorador @token_requerido: protege un endpoint para que solo lo usen
usuarios con un token JWT válido.

El cliente debe enviar el encabezado HTTP:
  Authorization: Bearer <access_token>

Si el token es válido, dejamos disponibles en `g` (objeto global de la petición):
  g.token -> el token tal cual (lo usaremos para hablar con Supabase)
  g.usuario -> los claims del token (sub = id del usuario, email, role, exp...)
"""
from functools import wraps

from flask import g, request
from flask_smorest import abort

from app.auth.jwt_utils import TokenInvalido, verificar_token

def _extraer_token() -> str:
    encabezado = request.headers.get("Authorization", "")
    partes = encabezado.split()
```

```

if len(partes) != 2 or partes[0].lower() != "bearer":
    abort(401, message="Falta el encabezado 'Authorization: Bearer <token>'")
return partes[1]

def token_requerido(funcion):
    @wraps(funcion)
    def envoltura(*args, **kwargs):
        token = _extraer_token()
        try:
            claims = verificar_token(token)
        except TokenInvalido as error:
            abort(401, message=str(error))

        g.token = token
        g.usuario = claims
        return funcion(*args, **kwargs)

    return envoltura

```

C.5 app/auth/schemas.py: esquemas de autenticación

Con **Marshmallow** describimos los datos de entrada y salida. flask-smorest los usa para **validar** (si falta el email responde 422 sin que escribamos un `if`) y para **documentar** en Swagger.

□ **Archivo:** app/auth/schemas.py

```

"""
schemas.py (auth)
-----
Esquemas de Marshmallow: definen QUÉ datos recibe y devuelve cada endpoint.
flask-smorest los usa para:
1. Validar el JSON de entrada (si falta un campo responde 422 automáticamente).
2. Generar la documentación Swagger/OpenAPI sin escribirla a mano.
"""
from marshmallow import Schema, fields, validate

class RegistroSchema(Schema):
    email = fields.Email(required=True, metadata={"example": "ana@ejemplo.com"})
    password = fields.Str(
        required=True,
        load_only=True,
        validate=validate.Length(min=6, error="La contraseña debe tener al menos 6
caracteres."),
        metadata={"example": "Clave123!"},
    )
    nombre = fields.Str(
        load_default=None,
        validate=validate.Length(max=80),
        metadata={"example": "Ana Pérez"},
    )

class LoginSchema(Schema):
    email = fields.Email(required=True, metadata={"example": "ana@ejemplo.com"})
    password = fields.Str(required=True, load_only=True, metadata={"example":

```

```

"Clave123!"})

class RefreshSchema(Schema):
    refresh_token = fields.Str(required=True, metadata={"description": "El refresh_token
recibido en el login."})

class UsuarioSchema(Schema):
    id = fields.Str()
    email = fields.Str()
    nombre = fields.Str(allow_none=True)

class SesionSchema(Schema):
    access_token = fields.Str(metadata={"description": "JWT que se envía en
'Authorization: Bearer ...'"})
    refresh_token = fields.Str(metadata={"description": "Sirve para obtener un nuevo
access_token."})
    token_type = fields.Str(metadata={"example": "bearer"})
    expires_in = fields.Int(metadata={"description": "Segundos de vida del
access_token."})
    usuario = fields.Nested(UsuarioSchema)

class RegistroRespuestaSchema(Schema):
    mensaje = fields.Str()
    usuario = fields.Nested(UsuarioSchema)
    sesion = fields.Nested(SesionSchema, allow_none=True)

class PerfilSchema(Schema):
    id = fields.Str(metadata={"description": "Claim 'sub' del JWT (id del usuario)."})
    email = fields.Str(allow_none=True)
    rol = fields.Str(metadata={"description": "Claim 'role' (authenticated)."})
    emitido_en = fields.Int(metadata={"description": "Claim 'iat' (fecha de emisión,
UNIX)."})
    expira_en = fields.Int(metadata={"description": "Claim 'exp' (fecha de expiración,
UNIX)."})
    segundos_restantes = fields.Int()
    claims = fields.Dict(metadata={"description": "Todos los claims del token
decodificado."})

```

C.6 app/auth/routes.py: Blueprint de autenticación

Registro, login, renovación del token y perfil. Observa los decoradores de cada ruta:

- `@blp.route(...)`: URL y método HTTP.
- `@blp.arguments(Schema)`: valida el JSON de entrada y lo entrega como parámetro `datos`.
- `@blp.response(código, Schema)`: formatea la respuesta y la documenta.
- `@blp.doc(security=...)`: muestra el candado  en Swagger.
- La **primera línea del docstring** es el título del endpoint en Swagger; el resto, su descripción.

❏ **Archivo:** app/auth/routes.py

```

"""
routes.py (auth)

```

Blueprint de AUTENTICACIÓN. Todas sus rutas empiezan con /api/auth

POST /api/auth/registro -> crea un usuario en Supabase Auth
POST /api/auth/login -> inicia sesión y devuelve el JWT (access_token)
POST /api/auth/refresh -> obtiene un nuevo access_token con el refresh_token
GET /api/auth/perfil -> (protegido) muestra lo que contiene tu JWT

Nota: La API NO guarda contraseñas. Supabase Auth se encarga de eso
(las almacena cifradas con bcrypt en el esquema auth).

"""

import time

from flask import g

from flask_smorest import Blueprint

from app.auth.decorators import token_requerido

from app.auth.schemas import (LoginSchema, PerfilSchema, RefreshSchema,
RegistroRespuestaSchema, RegistroSchema, SesionSchema)

from app.supabase_client import cliente_publico

blp = Blueprint(
 "auth", __name__,
 url_prefix="/api/auth",
 description="Registro, inicio de sesión y tokens JWT (Supabase Auth).",
)

def _usuario_a_dict(usuario) -> dict:

metadatos = getattr(usuario, "user_metadata", None) or {}

return {"id": usuario.id, "email": usuario.email, "nombre": metadatos.get("nombre")}

def _sesion_a_dict(sesion, usuario) -> dict:

return {
 "access_token": sesion.access_token,
 "refresh_token": sesion.refresh_token,
 "token_type": sesion.token_type,
 "expires_in": sesion.expires_in,
 "usuario": _usuario_a_dict(usuario),
 }

@blp.route("/registro", methods=["POST"])

@blp.arguments(RegistroSchema)

@blp.response(201, RegistroRespuestaSchema)

@blp.alt_response(409, description="El correo ya está registrado.")

def registrar(datos):

"""Registrar un nuevo usuario

Crea el usuario en Supabase Auth. Si en Supabase está desactivado
 "Confirm email", la respuesta incluye directamente la sesión (tokens).
 """

respuesta = cliente_publico().auth.sign_up({
 "email": datos["email"],
 "password": datos["password"],
 "options": {"data": {"nombre": datos.get("nombre")}}, # se guarda en
 user_metadata
 })

```

    if respuesta.session is None:
        mensaje = ("Usuario creado. Revisa tu correo para confirmar la cuenta "
                    "y luego inicia sesión en /api/auth/login.")
        sesion = None
    else:
        mensaje = "Usuario creado e inició sesión correctamente."
        sesion = _sesion_a_dict(respuesta.session, respuesta.user)

    return {"mensaje": mensaje, "usuario": _usuario_a_dict(respuesta.user), "sesion":
sesion}

@blp.route("/login", methods=["POST"])
@blp.arguments(LoginSchema)
@blp.response(200, SesionSchema)
@blp.alt_response(401, description="Correo o contraseña incorrectos.")
def iniciar_sesion(datos):
    """Iniciar sesión

    Devuelve un access_token (JWT). Cópialo y pégalo en el botón
    Authorize de Swagger para usar los endpoints protegidos.
    """
    respuesta = cliente_publico().auth.sign_in_with_password({
        "email": datos["email"],
        "password": datos["password"],
    })
    return _sesion_a_dict(respuesta.session, respuesta.user)

@blp.route("/refresh", methods=["POST"])
@blp.arguments(RefreshSchema)
@blp.response(200, SesionSchema)
@blp.alt_response(401, description="refresh_token inválido.")
def refrescar(datos):
    """Renovar el access_token

    Los access_token duran poco (por defecto 1 hora). Con el refresh_token
    obtienes uno nuevo sin volver a escribir la contraseña.
    """
    respuesta = cliente_publico().auth.refresh_session(datos["refresh_token"])
    return _sesion_a_dict(respuesta.session, respuesta.user)

@blp.route("/perfil", methods=["GET"])
@token_requerido
@blp.response(200, PerfilSchema)
@blp.alt_response(401, description="Token ausente, inválido o vencido.")
@blp.doc(security=[{"BearerAuth": []}])
def perfil():
    """Ver mi perfil (contenido del JWT)

    Endpoint protegido. Muestra los claims del token que enviaste;
    sirve para entender qué información viaja dentro de un JWT.
    """
    claims = g.usuario
    return {
        "id": claims.get("sub"),
        "email": claims.get("email"),
    }

```

```

        "rol": claims.get("role"),
        "emitido_en": claims.get("iat"),
        "expira_en": claims.get("exp"),
        "segundos_restantes": max(0, int(claims.get("exp", 0) - time.time())),
        "claims": claims,
    }

```

C.7 app/tareas/schemas.py: esquemas de tareas

□ Archivo: app/tareas/schemas.py

```

"""
schemas.py (tareas)
-----
Esquemas de validación y documentación para el recurso "tareas".

Observa que NINGÚN esquema de entrada acepta 'user_id': el dueño de la tarea
Lo pone PostgreSQL automáticamente con DEFAULT auth.uid().
Si alguien intenta enviar 'user_id', Marshmallow responde 422 (campo desconocido).
"""

from marshmallow import Schema, fields, validate

PRIORIDADES = ("baja", "media", "alta")

class TareaSchema(Schema):
    """Cómo se devuelve una tarea."""
    id = fields.UUID(dump_only=True)
    user_id = fields.UUID(dump_only=True, metadata={"description": "Dueño de la tarea (auth.uid())."})
    titulo = fields.Str()
    descripcion = fields.Str(allow_none=True)
    prioridad = fields.Str()
    completada = fields.Bool()
    creado_en = fields.Str(dump_only=True)

class TareaCrearSchema(Schema):
    """Datos para crear una tarea."""
    titulo = fields.Str(required=True, validate=validate.Length(min=3, max=120),
                        metadata={"example": "Estudiar RLS en Supabase"})
    descripcion = fields.Str(load_default=None, allow_none=True,
                             validate=validate.Length(max=500),
                             metadata={"example": "Leer la guía del laboratorio"})
    prioridad = fields.Str(load_default="media", validate=validate.OneOf(PRIORIDADES),
                           metadata={"example": "alta"})
    completada = fields.Bool(load_default=False, metadata={"example": False})

class TareaActualizarSchema(Schema):
    """Datos para actualizar una tarea. Todos los campos son opcionales."""
    titulo = fields.Str(validate=validate.Length(min=3, max=120), metadata={"example": "Título corregido"})
    descripcion = fields.Str(allow_none=True, validate=validate.Length(max=500))
    prioridad = fields.Str(validate=validate.OneOf(PRIORIDADES))
    completada = fields.Bool(metadata={"example": True})

class FiltroTareasSchema(Schema):

```

```
"""Filtros opcionales en la URL: /api/tareas?completada=true&prioridad=alta"""
completada = fields.Bool()
prioridad = fields.Str(validate=validate.OneOf(PRIORIDADES))
```

```
class MensajeSchema(Schema):
    mensaje = fields.Str()
    tarea = fields.Nested(TareaSchema)
```

❑ Por defecto Marshmallow **rechaza campos desconocidos**. Como `user_id` no está en `TareaCrearSchema`, si alguien lo envía recibe 422. Así nadie puede crear tareas “a nombre de otro”.

C.8 app/tareas/routes.py: Blueprint del CRUD

❑ **Archivo:** app/tareas/routes.py

```
"""
routes.py (tareas)
-----
Blueprint del CRUD de TAREAS. Todas las rutas empiezan con /api/tareas
y TODAS están protegidas con @token_requerido.

GET      /api/tareas           -> listar MIS tareas (filtros opcionales)
POST     /api/tareas           -> crear una tarea
GET      /api/tareas/<id>      -> ver una tarea
PATCH   /api/tareas/<id>      -> actualizar campos de una tarea
DELETE   /api/tareas/<id>      -> eliminar una tarea

📌 FÍJATE: en ninguna consulta escribimos .eq("user_id", ...).
¡No hace falta! Como la consulta viaja con el JWT del usuario, las
políticas RLS de PostgreSQL filtran solas las filas que le pertenecen.
Si el usuario B pide la tarea del usuario A, para PostgreSQL esa fila
"no existe" y respondemos 404.
"""

from flask import g
from flask_smorest import Blueprint, abort

from app.auth.decorators import token_requerido
from app.supabase_client import cliente_de_usuario
from app.tareas.schemas import (FiltroTareasSchema, MensajeSchema, TareaActualizarSchema,
                                TareaCrearSchema, TareaSchema)

TABLA = "tareas"
SEGURIDAD = [{"BearerAuth": []}]

blp = Blueprint(
    "tareas", __name__,
    url_prefix="/api/tareas",
    description="CRUD de tareas protegido con JWT y Row Level Security (RLS).",
)

def _tabla():
    """Acceso a la tabla usando el token del usuario actual (RLS activo)."""
    return cliente_de_usuario(g.token).table(TABLA)

@blp.route("", methods=["GET"])
```

```

@token_requerido
@blp.arguments(FiltroTareasSchema, location="query", as_kwargs=True)
@blp.response(200, TareaSchema(many=True))
@blp.alt_response(401, description="Token ausente, inválido o vencido.")
@blp.doc(security=SEGURIDAD)
def listar(completada=None, prioridad=None):
    """Listar mis tareas

    Devuelve SOLO las tareas del usuario dueño del token (gracias a RLS).
    Filtros opcionales: `?completada=true` y `?prioridad=alta`.
    """
    consulta = _tabla().select("*").order("creado_en", desc=True)
    if completada is not None:
        consulta = consulta.eq("completada", "true" if completada else "false")
    if prioridad is not None:
        consulta = consulta.eq("prioridad", prioridad)
    return consulta.execute().data

@blp.route("", methods=["POST"])
@token_requerido
@blp.arguments(TareaCrearSchema)
@blp.response(201, TareaSchema)
@blp.alt_response(401, description="Token ausente, inválido o vencido.")
@blp.alt_response(422, description="Datos inválidos.")
@blp.doc(security=SEGURIDAD)
def crear(datos):
    """Crear una tarea

    No envíes `user_id`: PostgreSQL lo completa con `auth.uid()`
    y la política RLS de INSERT verifica que seas tú.
    """
    resultado = _tabla().insert(datos).execute()
    return resultado.data[0]

@blp.route("/<uuid:tarea_id>", methods=["GET"])
@token_requerido
@blp.response(200, TareaSchema)
@blp.alt_response(404, description="No existe o no te pertenece.")
@blp.doc(security=SEGURIDAD)
def obtener(tarea_id):
    """Ver una tarea por su id"""
    resultado = _tabla().select("*").eq("id", str(tarea_id)).execute()
    if not resultado.data:
        abort(404, message="Tarea no encontrada (no existe o pertenece a otro usuario).")
    return resultado.data[0]

@blp.route("/<uuid:tarea_id>", methods=["PATCH"])
@token_requerido
@blp.arguments(TareaActualizarSchema)
@blp.response(200, TareaSchema)
@blp.alt_response(404, description="No existe o no te pertenece.")
@blp.doc(security=SEGURIDAD)
def actualizar(datos, tarea_id):
    """Actualizar una tarea

    Envía solo los campos que quieras cambiar, por ejemplo `{"completada": true}`.

```

```

"""
if not datos:
    abort(400, message="Envía al menos un campo para actualizar.")
resultado = _tabla().update(datos).eq("id", str(tarea_id)).execute()
if not resultado.data:
    abort(404, message="Tarea no encontrada (no existe o pertenece a otro usuario).")
return resultado.data[0]

@blp.route("/<uuid:tarea_id>", methods=["DELETE"])
@token_requerido
@blp.response(200, MensajeSchema)
@blp.alt_response(404, description="No existe o no te pertenece.")
@blp.doc(security=SEGURIDAD)
def eliminar(tarea_id):
    """Eliminar una tarea"""
    resultado = _tabla().delete().eq("id", str(tarea_id)).execute()
    if not resultado.data:
        abort(404, message="Tarea no encontrada (no existe o pertenece a otro usuario).")
    return {"mensaje": "Tarea eliminada.", "tarea": resultado.data[0]}

```

❑ **Observa con atención:** en ninguna consulta escribimos `.eq("user_id", ...)`. El filtro por usuario lo hace **PostgreSQL** gracias a RLS. Por eso, cuando alguien pide una tarea ajena, recibe `404`: para la base de datos esa fila *no existe* para él.

C.9 app/salud/routes.py: Blueprint de salud

❑ **Archivo:** app/salud/routes.py

```

"""
routes.py (salud)
-----
Blueprint de "salud" (health check). Sirve para saber si la API está viva.
Render y Railway lo pueden usar para comprobar que el despliegue funcionó.
"""
from datetime import datetime, timezone

from flask import current_app
from flask_smorest import Blueprint

blp = Blueprint("salud", __name__, url_prefix="/api", description="Estado del servicio.")

@blp.route("/salud", methods=["GET"])
@blp.response(200)
def salud():
    """Comprobar que la API está en línea"""
    return {
        "estado": "ok",
        "servicio": current_app.config["API_TITLE"],
        "version": current_app.config["API_VERSION"],
        "supabase_configurado": bool(current_app.config["SUPABASE_URL"] and
current_app.config["SUPABASE_KEY"]),
        "hora_servidor_utc": datetime.now(timezone.utc).isoformat(timespec="seconds"),
    }

```

C.10 app/errores.py: manejo de errores

Traduce las excepciones de Supabase a respuestas JSON claras y en español (por ejemplo, credenciales incorrectas → 401, correo repetido → 409, bloqueo de RLS → 403).

❏ **Archivo:** app/errores.py

```
"""
errores.py
-----
Manejadores globales de errores. Convierten las excepciones de Supabase
en respuestas JSON claras, con el mismo formato que usa flask-smorest:

    {"code": 404, "status": "Not Found", "message": "..."}
"""

import logging
from http import HTTPStatus

import httpx
from flask import jsonify
from postgrest.exceptions import APIError
from supabase_auth.errors import AuthAPIError, AuthError

log = logging.getLogger(__name__)

# Traducción de los mensajes más comunes de Supabase Auth.
TRADUCCIONES_AUTH = {
    "invalid login credentials": (401, "Correo o contraseña incorrectos."),
    "email not confirmed": (403, "Debes confirmar tu correo antes de iniciar sesión "
                              "(o desactiva 'Confirm email' en Supabase para el "
                              "laboratorio)."),
    "user already registered": (409, "Ya existe un usuario con ese correo. Usa "
                                     "/api/auth/login."),
    "invalid refresh token": (401, "El refresh_token no es válido o ya fue usado."),
}

def respuesta_error(codigo: int, mensaje: str, **extra):
    cuerpo = {"code": codigo, "status": HTTPStatus(codigo).phrase, "message": mensaje}
    cuerpo.update(extra)
    return jsonify(cuerpo), codigo

def registrar_manejadores_errores(app):
    @app.errorhandler(AuthAPIError)
    def error_auth_api(error: AuthAPIError):
        texto = (error.message or "").lower()
        for clave, (codigo, mensaje) in TRADUCCIONES_AUTH.items():
            if clave in texto:
                return respuesta_error(codigo, mensaje)
        codigo = error.status if 400 <= (error.status or 0) < 500 else 400
        return respuesta_error(codigo, f"Supabase Auth: {error.message}")

    @app.errorhandler(AuthError)
    def error_auth(error: AuthError):
        return respuesta_error(400, f"Supabase Auth: {error}")

    @app.errorhandler(APIError)
```

```

def error_postgres(error: APIError):
    # Códigos de PostgreSQL / PostgREST más frecuentes en el Laboratorio.
    if error.code == "42501": # insufficient_privilege: una política RLS lo impidió
        return respuesta_error(403, "Operación bloqueada por Row Level Security
(RLS).",
                                detalle=error.message)
    if error.code == "PGRST301" or error.code == "PGRST303": # JWT inválido o
vencido
        return respuesta_error(401, "Supabase rechazó el token JWT.",
detalle=error.message)
    if error.code == "PGRST205" or error.code == "42P01": # La tabla no existe
        return respuesta_error(500, "La tabla 'tareass' no existe. Ejecuta
sql/01_esquema_y_qls.sql "
                                "en el SQL Editor de Supabase.",
detalle=error.message)
    if error.code == "22P02": # texto con formato inválido (por ejemplo, un UUID
mal escrito)
        return respuesta_error(400, "Algún valor enviado tiene un formato inválido.",
detalle=error.message)
    if error.code == "23514": # check_violation
        return respuesta_error(422, "Los datos no cumplen una restricción de la
tabla.",
                                detalle=error.message)
    log.warning("Error de PostgREST: %r", error)
    return respuesta_error(400, "Error al consultar la base de datos.",
detalle=error.message, codigo_pg=error.code)

@app.errorhandler(httpx.HTTPError)
def error_red(error):
    log.error("No se pudo conectar con Supabase: %s", error)
    return respuesta_error(503, "No se pudo conectar con Supabase. Revisa
SUPABASE_URL "
                                "y tu conexión a internet.")

```

C.11 app/__init__.py: la fábrica de la aplicación

Une todas las piezas: configuración, CORS, Swagger, **registro de los Blueprints** y manejo de errores. También define el esquema de seguridad **BearerAuth**, que activa el botón **Authorize** en Swagger.

❑ **Archivo:** app/__init__.py

```

"""
app/__init__.py
-----
"Fábrica de aplicaciones" (Application Factory): La función create_app()
construye y configura la aplicación Flask.

Aquí se:
1. Carga la configuración.
2. Activa CORS (para que un frontend en otro dominio pueda consumir la API).
3. Crea el objeto Api de flask-smorest (genera OpenAPI + Swagger UI).
4. Registra los Blueprints (cada módulo de la API).
5. Registra los manejadores de errores.
"""
import logging

from flask import Flask, redirect

```

```

from flask_cors import CORS
from flask_smorest import Api

from app.config import Config

def create_app(config_object=Config) -> Flask:
    app = Flask(__name__)
    app.config.from_object(config_object)
    app.json.sort_keys = False
    app.json.ensure_ascii = False # muestra tildes y ñ sin escaparlas
    logging.basicConfig(level=logging.INFO)

    # Si faltan las variables de Supabase avisamos claramente y no arrancamos.
    faltantes = [v for v in ("SUPABASE_URL", "SUPABASE_KEY") if not app.config.get(v)]
    if faltantes:
        raise RuntimeError(
            f"Faltan variables de entorno: {'', '.join(faltantes)}".
            "Copia .env.example a .env y complétalo (o configúralas en Render/Railway)."
        )

    CORS(app)

    api = Api(app)
    # Esquema de seguridad "BearerAuth": habilita el botón Authorize en Swagger.
    api.spec.components.security_scheme(
        "BearerAuth",
        {"type": "http", "scheme": "bearer", "bearerFormat": "JWT",
         "description": "Pega aquí el access_token obtenido en /api/auth/login"},
    )

    # Importamos aquí los Blueprints para evitar importaciones circulares.
    from app.auth.routes import blp as auth_blp
    from app.salud.routes import blp as salud_blp
    from app.tareas.routes import blp as tareas_blp

    api.register_blueprint(salud_blp)
    api.register_blueprint(auth_blp)
    api.register_blueprint(tareas_blp)

    from app.errores import registrar_manejadores_errores
    registrar_manejadores_errores(app)

    @app.get("/")
    def inicio():
        # La raíz redirige a la documentación Swagger.
        return redirect("/docs")

    return app

```

C.12 wsgi.py: punto de entrada

□ Archivo: wsgi.py

```

"""
wsgi.py
-----
Punto de entrada de la aplicación.

```

```

* En tu computadora: python wsgi.py -> http://localhost:8000/docs
* En producción:    gunicorn wsgi:app (Lo usan Render y Railway)
"""
import os

from app import create_app

app = create_app()

if __name__ == "__main__":
    puerto = int(os.getenv("PORT", "8000"))
    app.run(host="0.0.0.0", port=puerto, debug=True)

```

C.13 Archivos de despliegue

Procfile (Railway y otros servicios lo leen para saber cómo iniciar la app):

□ **Archivo:** Procfile

```
web: gunicorn wsgi:app --bind 0.0.0.0:$PORT --workers 2 --timeout 120
```

railway.json:

□ **Archivo:** railway.json

```

{
  "$schema": "https://railway.com/railway.schema.json",
  "deploy": {
    "startCommand": "gunicorn wsgi:app --bind 0.0.0.0:$PORT --workers 2 --timeout 120",
    "healthcheckPath": "/api/salud"
  }
}

```

render.yaml (opcional, permite crear el servicio en Render con un clic usando *Blueprint*):

□ **Archivo:** render.yaml

```

# Blueprint de Render (opcional): New > Blueprint > selecciona tu repositorio.
services:
  - type: web
    name: api-tareas-flask
    runtime: python
    plan: free
    buildCommand: pip install -r requirements.txt
    startCommand: gunicorn wsgi:app --bind 0.0.0.0:$PORT --workers 2 --timeout 120
    healthCheckPath: /api/salud
    envVars:
      - key: PYTHON_VERSION
        value: "3.12.8"
      - key: SUPABASE_URL
        sync: false
      - key: SUPABASE_KEY
        sync: false
      - key: SUPABASE_JWT_SECRET
        sync: false

```

.python-version (indica la versión de Python a Render y Railway):

□ **Archivo:** .python-version

3.12

pytest.ini:

❑ **Archivo:** pytest.ini

```
[pytest]
pythonpath = .
testpaths = tests
```

Parte D: ejecutar y probar con Swagger

D.1 Iniciar la API

Con el entorno virtual activo y el .env completo:

```
python wsgi.py
```

Verás algo como `Running on http://127.0.0.1:8000`. Abre en el navegador:

- <http://localhost:8000/api/salud> → debe responder "estado": "ok".
- <http://localhost:8000/docs> → documentación **Swagger UI**.

D.2 Recorrido guiado en Swagger

1. **Registrar:** abre `POST /api/auth/registro` → **Try it out** → cambia el JSON y pulsa **Execute**:

```
{ "email": "ana@ejemplo.com", "password": "Clave123!", "nombre": "Ana" }
```

Respuesta 201, con `sesion.access_token`.

2. **Iniciar sesión:** `POST /api/auth/login` con el mismo email y contraseña. Copia el valor de `access_token` (sin comillas).
3. **Autorizar:** pulsa el botón verde **Authorize** ☐ (arriba a la derecha), pega el token (sin escribir `Bearer`), pulsa **Authorize** y luego **Close**. Los candados se cierran ☐.
4. **Ver tu JWT:** ejecuta `GET /api/auth/perfil`. Verás tu `id` (claim `sub`), `email`, `rol` y cuántos segundos le quedan al token.
5. **Crear tareas:** `POST /api/tareas`:

```
{ "titulo": "Estudiar RLS", "descripcion": "Leer la guía", "prioridad": "alta" }
```

6. **Listar:** `GET /api/tareas`. Prueba los filtros `completada` y `prioridad`.
7. **Actualizar:** `PATCH /api/tareas/{tarea_id}` con el `id` de una tarea y el cuerpo `{ "completada": true }`.
8. **Eliminar:** `DELETE /api/tareas/{tarea_id}`.

D.4 Probar con **curl** (opcional)

```
# Login y guardar el token (macOS/Linux)
TOKEN=$(curl -s -X POST http://localhost:8000/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"ana@ejemplo.com","password":"Clave123!"}' | python3 -c "import
sys,json;print(json.load(sys.stdin)['access_token'])")

# Crear una tarea
curl -X POST http://localhost:8000/api/tareas \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"titulo":"Tarea desde curl","prioridad":"baja"}'

# Listar
curl http://localhost:8000/api/tareas -H "Authorization: Bearer $TOKEN"
```

D.5 Pruebas automáticas

El proyecto incluye pruebas que **no necesitan internet** (simulan Supabase):

❑ **Archivo:** tests/test_api.py

```
"""
Pruebas automáticas (no necesitan internet ni Supabase: todo se simula).

Ejecutar:  pytest -v
"""

import time
import types

import jwt
import pytest
from cryptography.hazmat.primitives.asymmetric import ec

from app import create_app
from app.auth import jwt_utils
from app.config import TestConfig

URL = TestConfig.SUPABASE_URL
SECRETO = TestConfig.SUPABASE_JWT_SECRET
USUARIO_ID = "11111111-1111-1111-1111-111111111111"

def crear_token(**cambios) -> str:
    ahora = int(time.time())
    claims = {"sub": USUARIO_ID, "email": "ana@ejemplo.com", "role": "authenticated",
              "aud": "authenticated", "iss": f"{URL}/auth/v1", "iat": ahora, "exp": ahora
+ 3600}
    claims.update(cambios)
    return jwt.encode(claims, SECRETO, algorithm="HS256")

@pytest.fixture
def cliente():
    app = create_app(TestConfig)
    return app.test_client()

# ----- Simulación de Supabase
class ConsultaFalsa:
```

```

"""Imita la cadena de PostgREST: .select().eq().order().execute()"""

def __init__(self, registro, datos):
    self.registro, self.datos = registro, datos

def __getattr__(self, nombre):
    def metodo(*args, **kwargs):
        self.registro.append((nombre, args))
        return self
    return metodo

def execute(self):
    return types.SimpleNamespace(data=self.datos)

@pytest.fixture
def supabase_falso(monkeypatch):
    estado = {"token": None, "llamadas": [], "datos": []}

    def cliente_de_usuario(token):
        estado["token"] = token
        return types.SimpleNamespace(table=lambda nombre:
ConsultaFalsa(estado["llamadas"], estado["datos"]))

    monkeypatch.setattr("app.tareas.routes.cliente_de_usuario", cliente_de_usuario)
    return estado

# ----- Pruebas
def test_salud(cliente):
    r = cliente.get("/api/salud")
    assert r.status_code == 200
    assert r.get_json()["estado"] == "ok"

def test_swagger_y_openapi(cliente):
    assert cliente.get("/docs").status_code == 200
    spec = cliente.get("/openapi.json").get_json()
    assert "BearerAuth" in spec["components"]["securitySchemes"]
    assert "/api/tareas" in spec["paths"]

def test_sin_token_da_401(cliente):
    assert cliente.get("/api/tareas").status_code == 401

def test_token_valido_hs256(cliente):
    r = cliente.get("/api/auth/perfil", headers={"Authorization": f"Bearer
{crear_token()}"})
    assert r.status_code == 200
    assert r.get_json()["id"] == USUARIO_ID

@pytest.mark.parametrize("cambios", [
    {"exp": int(time.time()) - 3600},          # vencido
    {"aud": "anon"},                          # audiencia incorrecta
    {"iss": "https://otro.supabase.co/auth/v1"}, # otro proyecto
])
def test_tokens_invalidos(cliente, cambios):
    r = cliente.get("/api/auth/perfil", headers={"Authorization": f"Bearer

```

```

{crear_token(**cambios)}}))
    assert r.status_code == 401

def test_firma_alterada(cliente):
    token = crear_token()
    alterado = token[:-3] + ("aaa" if not token.endswith("aaa") else "bbb")
    r = cliente.get("/api/auth/perfil", headers={"Authorization": f"Bearer {alterado}"})
    assert r.status_code == 401

def test_token_es256_con_jwks(cliente, monkeypatch):
    clave_privada = ec.generate_private_key(ec.SEC256R1())
    ahora = int(time.time())
    token = jwt.encode({"sub": USUARIO_ID, "aud": "authenticated", "iss":
f"{URL}/auth/v1",
                        "role": "authenticated", "iat": ahora, "exp": ahora + 60},
                        clave_privada, algorithm="ES256", headers={"kid": "k1"})
    jwks_falso = types.SimpleNamespace(
        get_signing_key_from_jwt=lambda t:
types.SimpleNamespace(key=clave_privada.public_key()))
    monkeypatch.setattr(jwt_utils, "_cliente_jwks", lambda: jwks_falso)
    r = cliente.get("/api/auth/perfil", headers={"Authorization": f"Bearer {token}"})
    assert r.status_code == 200

def test_listar_usa_el_token_del_usuario(cliente, supabase_falso):
    supabase_falso["datos"] = []
    token = crear_token()
    r = cliente.get("/api/tareas?completada=true", headers={"Authorization": f"Bearer
{token}"})
    assert r.status_code == 200
    assert supabase_falso["token"] == token # RLS usará este JWT
    assert ("eq", ("completada", "true")) in supabase_falso["llamadas"]
    assert not any(a and a[0] == "user_id" for _, a in supabase_falso["llamadas"]) # sin
filtro manual

def test_crear_no_acepta_user_id(cliente, supabase_falso):
    r = cliente.post("/api/tareas", json={"titulo": "Hola", "user_id": USUARIO_ID},
                    headers={"Authorization": f"Bearer {crear_token()}"})
    assert r.status_code == 422

def test_obtener_tarea ajena_da_404(cliente, supabase_falso):
    supabase_falso["datos"] = [] # RLS no devuelve filas de otros usuarios
    r = cliente.get("/api/tareas/22222222-2222-2222-2222-222222222222",
                    headers={"Authorization": f"Bearer {crear_token()}"})
    assert r.status_code == 404

```

Ejecútalas:

```
pytest -v
```

Resultado esperado: 12 passed.

Parte E: demostrar RLS (usuario A vs. usuario B)

E.1 Demostración manual en Swagger

1. Registra a **Beto** (beto@ejemplo.com) y copia su token.
2. Con el token de **Ana**, crea una tarea y copia su **id**.
3. Pulsa **Authorize**, haz **Logout** y autoriza con el token de **Beto**.
4. Como Beto: **GET** `/api/tareas` → la tarea de Ana **no aparece**.
5. Como Beto: **GET**, **PATCH** y **DELETE** sobre `/api/tareas/{id de Ana}` → **404**.
6. En Supabase → **Table Editor** → **tareas**: allí sí ves **todas** las filas, porque el panel usa un rol administrador que ignora RLS.

E.2 Demostración automática con un script

□ **Archivo:** `scripts/demo_rls.py`

```
"""
demo_rls.py
-----
Demostración automática de JWT + RLS contra TU API (local o publicada).

Crea (o reutiliza) dos usuarios, Ana y Beto, y comprueba que:
1. Sin token no se puede entrar (401).
2. Un token alterado es rechazado (401).
3. Ana crea una tarea y la ve.
4. Beto NO ve la tarea de Ana, ni puede leerla, editarla o borrarla (404).
5. Nadie puede asignarse un user_id ajeno (422).

Uso:
    python scripts/demo_rls.py                                # API Local
(http://localhost:8000)
    python scripts/demo_rls.py https://mi-api.onrender.com    # API publicada

Requisito: en Supabase debe estar DESACTIVADO "Confirm email"
(Authentication > Sign In / Providers > Email).
"""
import sys

import httpx

BASE = (sys.argv[1] if len(sys.argv) > 1 else "http://localhost:8000").rstrip("/")
CLAVE = "Laboratorio123!"
USUARIOS = {"Ana": "ana.demo.rls@ejemplo.com", "Beto": "beto.demo.rls@ejemplo.com"}

VERDE, ROJO, AZUL, FIN = "\033[92m", "\033[91m", "\033[94m", "\033[0m"
errores = 0

cliente = httpx.Client(base_url=BASE, timeout=90) # 90 s: Render gratis tarda en
"despertar"

def comprobar(descripcion: str, condicion: bool, detalle: str = ""):
    global errores
    if condicion:
        print(f" {VERDE}✓ {descripcion}{FIN}")
    else:
        errores += 1
```

```

        print(f" {ROJO}X {descripcion}{FIN} {detalle}")

def cabecera(token: str) -> dict:
    return {"Authorization": f"Bearer {token}"}

def obtener_token(nombre: str, email: str) -> str:
    """Inicia sesión; si el usuario no existe, lo registra."""
    r = cliente.post("/api/auth/login", json={"email": email, "password": CLAVE})
    if r.status_code == 200:
        return r.json()["access_token"]
    r = cliente.post("/api/auth/registro", json={"email": email, "password": CLAVE,
"nombre": nombre})
    datos = r.json()
    if r.status_code != 201 or not datos.get("sesion"):
        print(f"ROJO>No se pudo registrar a {nombre}: {r.status_code} {datos}{FIN}")
        print("¿Desactivaste 'Confirm email' en Supabase?")
        sys.exit(1)
    return datos["sesion"]["access_token"]

print(f"{AZUL}== Probando la API en {BASE} =={FIN}")
r = cliente.get("/api/salud")
comprobar("La API responde en /api/salud", r.status_code == 200, r.text)

print(f"\n{AZUL}1) Autenticación JWT{FIN}")
r = cliente.get("/api/tareas")
comprobar("Sin token -> 401", r.status_code == 401, f"(recibido {r.status_code})")

token_ana = obtener_token("Ana", USUARIOS["Ana"])
token_beto = obtener_token("Beto", USUARIOS["Beto"])
comprobar("Ana y Beto obtuvieron su access_token", bool(token_ana and token_beto))

token_alterado = token_ana[:-4] + ("AAAA" if not token_ana.endswith("AAAA") else "BBBB")
r = cliente.get("/api/tareas", headers=cabecera(token_alterado))
comprobar("Token alterado (firma inválida) -> 401", r.status_code == 401, f"(recibido
{r.status_code})")

r = cliente.get("/api/auth/perfil", headers=cabecera(token_ana))
comprobar("GET /api/auth/perfil muestra el email de Ana",
    r.status_code == 200 and r.json().get("email") == USUARIOS["Ana"], r.text)

print(f"\n{AZUL}2) Ana crea una tarea{FIN}")
r = cliente.post("/api/tareas", headers=cabecera(token_ana),
    json={"titulo": "Tarea secreta de Ana", "prioridad": "alta"})
comprobar("Ana crea la tarea -> 201", r.status_code == 201, r.text)
tarea = r.json() if r.status_code == 201 else {}
id_tarea = tarea.get("id", "00000000-0000-0000-0000-000000000000")

r = cliente.get("/api/tareas", headers=cabecera(token_ana))
comprobar("Ana ve su tarea en el listado", any(t["id"] == id_tarea for t in r.json()),
r.text)

print(f"\n{AZUL}3) Beto intenta acceder a la tarea de Ana (RLS){FIN}")
r = cliente.get("/api/tareas", headers=cabecera(token_beto))
comprobar("Beto NO la ve en su listado", all(t["id"] != id_tarea for t in r.json()),
r.text)

```

```

r = cliente.get(f"/api/tareas/{id_tarea}", headers=cabecera(token_beto))
comprobar("Beto NO puede leerla -> 404", r.status_code == 404, f"(recibido {r.status_code})")

r = cliente.patch(f"/api/tareas/{id_tarea}", headers=cabecera(token_beto),
json={"titulo": "Hackeada"})
comprobar("Beto NO puede editarla -> 404", r.status_code == 404, f"(recibido {r.status_code})")

r = cliente.delete(f"/api/tareas/{id_tarea}", headers=cabecera(token_beto))
comprobar("Beto NO puede borrarla -> 404", r.status_code == 404, f"(recibido {r.status_code})")

r = cliente.post("/api/tareas", headers=cabecera(token_beto),
                json={"titulo": "Tarea a nombre de Ana", "user_id":
tarea.get("user_id")})
comprobar("Beto NO puede enviar un user_id ajeno -> 422", r.status_code == 422,
f"(recibido {r.status_code})")

print(f"\n{AZUL}4) Ana sigue teniendo el control{FIN}")
r = cliente.patch(f"/api/tareas/{id_tarea}", headers=cabecera(token_ana),
json={"completada": True})
comprobar("Ana la marca como completada -> 200",
          r.status_code == 200 and r.json().get("completada") is True, r.text)

r = cliente.delete(f"/api/tareas/{id_tarea}", headers=cabecera(token_ana))
comprobar("Ana la elimina -> 200", r.status_code == 200, r.text)

print()
if errores == 0:
    print(f"{VERDE}🎉 ¡Todo correcto! JWT y RLS funcionan.{FIN}")
else:
    print(f"{ROJO}Hubo {errores} comprobación(es) con error.{FIN}")
sys.exit(1 if errores else 0)

```

Con la API corriendo en otra terminal:

```

python scripts/demo_rls.py # contra tu API local
python scripts/demo_rls.py https://TU-API.onrender.com # contra tu API publicada

```

Salida esperada:

```

== Probando la API en http://localhost:8000 ==
  ✓ La API responde en /api/salud

1) Autenticación JWT
  ✓ Sin token -> 401
  ✓ Ana y Beto obtuvieron su access_token
  ✓ Token alterado (firma inválida) -> 401
  ✓ GET /api/auth/perfil muestra el email de Ana

2) Ana crea una tarea
  ✓ Ana crea la tarea -> 201
  ✓ Ana ve su tarea en el listado

3) Beto intenta acceder a la tarea de Ana (RLS)
  ✓ Beto NO la ve en su listado
  ✓ Beto NO puede leerla -> 404
  ✓ Beto NO puede editarla -> 404

```

- ✓ Beto NO puede borrarla -> 404
- ✓ Beto NO puede enviar un user_id ajeno -> 422

4) Ana sigue teniendo el control

- ✓ Ana la marca como completada -> 200
- ✓ Ana la elimina -> 200

📌 ¡Todo correcto! JWT y RLS funcionan.

E.3 Ver RLS directamente en SQL (opcional)

Ejecuta **bloque por bloque** en el SQL Editor:

📄 **Archivo:** sql/02_probar_rls_en_sql_editor.sql

```
-- =====
-- 02_probar_rls_en_sql_editor.sql  (OPCIONAL, para entender RLS)
--
-- El SQL Editor de Supabase se ejecuta como el rol "postgres", que
-- IGNORA RLS (por eso ahí ves TODAS las tareas). Aquí "fingimos" ser
-- un usuario autenticado para ver cómo RLS filtra las filas.
--
-- Requisito: haber creado al menos 2 usuarios y algunas tareas con la API.
-- Ejecuta cada BLOQUE por separado (seleccionalo y pulsa Run).
-- =====

-- BLOQUE A: como postgres (sin RLS) -> ves las tareas de TODOS.
select t.titulo, u.email as dueño
from public.tareas t join auth.users u on u.id = t.user_id
order by u.email;

-- BLOQUE B: como el PRIMER usuario registrado -> solo ves SUS tareas.
begin;
select set_config(
    'request.jwt.claims',
    (select json_build_object('sub', id, 'role', 'authenticated')::text
     from auth.users order by created_at limit 1),
    true);
set local role authenticated;
select auth.uid() as soy_yo, titulo, user_id from public.tareas;
rollback;

-- BLOQUE C: como el SEGUNDO usuario registrado -> solo ves SUS tareas.
begin;
select set_config(
    'request.jwt.claims',
    (select json_build_object('sub', id, 'role', 'authenticated')::text
     from auth.users order by created_at offset 1 limit 1),
    true);
set local role authenticated;
select auth.uid() as soy_yo, titulo, user_id from public.tareas;
rollback;

-- BLOQUE D: como anónimo (sin token) -> error "permission denied".
begin;
set local role anon;
select * from public.tareas;
rollback;
```

E.4 Experimento: ¿qué pasa si desactivo RLS?

En el SQL Editor ejecuta `alter table public.tareas disable row level security;` y repite el paso E.1: **Beto verá las tareas de Ana**. Esto demuestra que la protección está en la base de datos. **Vuelve a activarlo** de inmediato:

```
alter table public.tareas enable row level security;
```

Parte F: publicar gratis en Render (o Railway)

F.1 Subir el proyecto a GitHub

```
git init
git add .
git commit -m "API REST Flask + Supabase con RLS, JWT y Swagger"
git branch -M main
git remote add origin https://github.com/TU-USUARIO/api-tareas-flask.git
git push -u origin main
```

Comprueba en GitHub que **no** aparece el archivo `.env`. Si apareció, bórralo del repositorio y **regenera** tus claves en Supabase.

F.2 Opción recomendada: Render (gratis, sin tarjeta)

	Render (Free)	Railway (Trial)
Costo	USD 0 permanente	USD 5 de crédito una sola vez ; luego plan Hobby de pago
Tarjeta	No	No para la prueba
Limitación	Se “duerme” tras 15 min sin tráfico; despierta en ~1 min	Se detiene al agotar el crédito
Horas	750 h/mes por cuenta	Según consumo

Pasos en Render:

1. Entra a <https://dashboard.render.com> e inicia sesión con GitHub.
2. **New** → **Web Service** → conecta tu repositorio `api-tareas-flask`.
3. Configura:
 - **Language:** Python 3
 - **Branch:** `main`
 - **Build Command:** `pip install -r requirements.txt`
 - **Start Command:** `gunicorn wsgi:app --bind 0.0.0.0:$PORT --workers 2 --timeout 120`
 - **Instance Type:** Free
4. En **Environment Variables** agrega `SUPABASE_URL` y `SUPABASE_KEY` (y `SUPABASE_JWT_SECRET` solo si lo usas).
5. Pulsa **Deploy Web Service** y espera a que el registro (*log*) muestre `Your service is live` 🟢.
6. Abre <https://TU-SERVICIO.onrender.com/docs> y repite el recorrido de la Parte D.
7. Ejecuta `python scripts/demo_rls.py https://TU-SERVICIO.onrender.com`.

❑ Si la API no se usó en 15 minutos, la primera petición tarda cerca de un minuto: es normal en el plan gratuito.

Alternativa: **New** → **Blueprint** detecta el archivo `render.yaml` y crea el servicio con esa configuración; solo tendrás que completar las variables.

F.3 Alternativa: Railway

1. Entra a <https://railway.com> con GitHub (recibes un crédito de prueba único de USD 5).
2. **New Project** → **Deploy from GitHub repo** → elige tu repositorio.
3. En el servicio → **Variables** → agrega `SUPABASE_URL` y `SUPABASE_KEY`.
4. **Settings** → **Networking** → **Generate Domain**.
5. Railway usa `railway.json/Procfile` para iniciar con Gunicorn. Abre <https://TU-DOMINIO.up.railway.app/docs>.

Errores frecuentes y soluciones

Síntoma	Causa probable	Solución
<code>RuntimeError: Faltan variables de entorno</code>	No existe <code>.env</code> o faltan valores	Copia <code>.env.example</code> a <code>.env</code> y complétalo; en Render, revisa <i>Environment</i>
<code>403</code> Debes confirmar tu correo al hacer login	<i>Confirm email</i> activado	Paso A.2
<code>500</code> La tabla 'tareas' no existe	No se ejecutó el SQL	Paso A.3
<code>401</code> Supabase rechazó el token o La firma del token no es válida	Token de otro proyecto, copiado incompleto o vencido	Vuelve a hacer login y copia el token completo
<code>401</code> El token expiró	Pasó 1 hora	<code>POST /api/auth/refresh</code> o login de nuevo
<code>503</code> No se pudo conectar con Supabase	<code>SUPABASE_URL</code> mal escrita o sin internet	Revisa la URL (sin / final)
<code>429</code> o <i>rate limit</i> al registrar	Demasiados registros seguidos	Espera unos minutos
<code>ModuleNotFoundError: No module named 'app'</code>	Ejecutas desde otra carpeta	Ejecuta los comandos en la raíz del proyecto
En macOS, el puerto 5000 está ocupado	AirPlay usa el 5000	Este proyecto usa el 8000
Render: <code>gunicorn: command not found</code>	Falta en <code>requirements.txt</code>	Verifica que esté y vuelve a desplegar
Swagger muestra <code>Failed to fetch</code> en Render	El servicio estaba dormido	Espera un minuto y reintenta

Retos para practicar

1. **Nuevo campo:** agrega `fecha_limite date` a la tabla y a los esquemas; permite filtrar `?vencidas=true`.
2. **Nuevo Blueprint:** crea `categorias` (tabla + RLS + CRUD) y relaciónala con `tareas`.
3. **Tareas compartidas:** crea una política SELECT extra que permita ver tareas marcadas como `publica = true`.
4. **Paginación:** agrega `?pagina=1&por_pagina=10` usando `.range(desde, hasta)`.
5. **Rol administrador:** usa `app_metadata` del JWT para permitir que un "admin" vea todas las tareas (pista: política con `(auth.jwt() -> 'app_metadata' -> 'rol') = 'admin'`).
6. **Pruebas:** agrega una prueba en `tests/test_api.py` para `PATCH` y `DELETE`.

Entregables del laboratorio

- ☐ Enlace al repositorio en GitHub (sin el archivo `.env`).
- ☐ Enlace público a `/docs` de tu API en Render o Railway.

- ☐ Captura de `python scripts/demo_rls.py <tu-url>` con todas las comprobaciones en verde.
- ☐ Captura de `pytest -v` con `12 passed`.
- ☐ Respuesta breve (3 a 5 líneas): *¿Por qué Beto recibe 404 y no 403 al pedir la tarea de Ana? ¿Qué ventaja de seguridad tiene esto?*